

Guía Exhaustiva de Modelos de Datos (Mongoose Schemas, Types e Interfaces)

Este documento detalla la arquitectura de backend de la capa de **Modelos** (`src/api/models/`), sus esquemas de Mongoose, índices de base de datos, tipados de TypeScript y las reglas de negocio críticas que rigen la persistencia de datos del sistema **API Costeo**.

Mapa General de la Capa de Persistencia (MongoDB & Mongoose)

La persistencia de datos está estructurada modularmente en **22 directorios** dentro de `src/api/models/`. Cada subdirectorio encapsula un dominio de negocio, agrupando el esquema de Mongoose (`*.model.ts`) y sus respectivas declaraciones de tipo y DTOs (`*.types.ts` o `type.ts`).

Mecanismos de Conexión Dinámica y Sucursales

El sistema no utiliza una única conexión estática para todos los datos operativos. La sucursal u oficina (`idOffice`) del usuario determina qué base de datos y qué perfil de almacenamiento se instanciarán de manera dinámica mediante el controlador de conexiones (`ConnectionModels`). Esto garantiza el aislamiento físico de la información operativa entre diferentes sucursales.

Patrones Transversales y Reglas de Negocio Globales

1. Esquema Común de Auditoría (`RegisterInfo/saveInfo.ts`)

Para cumplir con los estándares de control interno de Nicolás / Cicsa, casi todas las colecciones principales almacenan la identidad del usuario responsable de crear y modificar el documento.

- **Campos:** `createUserInfor` y `updateUserInfor`.
- **Estructura:**

```
export interface userInfo {  
  userId: string;  
  userName: string;  
  emailUser: string;  
}
```

- **Proceso:** Estos objetos se extraen del token JWT descifrado en el middleware de autenticación (`authMiddleware`) y se propagan automáticamente hacia los servicios para registrarse en la base de datos al guardar o actualizar documentos.

2. Generación Atómica de Folios Secuenciales (`SequenceFolioModels`)

Para evitar colisiones por concurrencia (dos capturas que obtengan el mismo ID numérico al mismo tiempo), el sistema implementa una colección dedicada a contadores atómicos llamada `SequenceFolio`.

- **Pre-save Hook:** En colecciones críticas como `Report` y `FictitiousTeb`, se ejecuta un middleware de Mongoose antes de insertar el documento (`doc.isNew`):

```
const counter = await SequenceFolioModel.findOneAndUpdate(  
  { table: 'NombreDeLaColeccion' },  
  { $inc: { folio: 1 } },  
  { new: true, upsert: true }  
);  
doc.folio = counter.folio;
```

- Esto garantiza la unicidad absoluta de folios secuenciales sin depender de auto-incrementos nativos de bases de datos relacionales.

3. Regla del 1.0 (Distribución y Ajuste de Decimales)

En la lógica diaria de cobro de activos, una jornada diaria de trabajo o mantenimiento debe sumar exactamente `1.00`.

- **Distribución:** Si un activo realiza múltiples actividades en un día, el tiempo se convierte a fracciones decimales.
- **Redondeo de Ajuste:** Debido a la imprecisión del redondeo matemático estándar, el sistema utiliza el **Método del Resto Mayor** (Largest Remainder Method) para prorratear los decimales sobrantes. Para saber dónde aplicar la diferencia, el esquema `JornDay` define una propiedad booleana `isAdjustment: true` en sus slots horarios. El sub-turno que tenga este flag activo absorberá el residuo matemático sobrante, asegurando que la suma total sea exactamente `1.00`.

4. Ciclo de Vida Limpio de Datos (Índices TTL - Time To Live)

Ciertas operaciones de captura, digitalización OCR e inteligencia artificial generan un alto volumen de datos transitorios. Para evitar el crecimiento desmedido de la base de datos, MongoDB elimina automáticamente los registros obsoletos mediante índices TTL en tres colecciones clave:

- **DailyReportScannerSession**: Se auto-elimina a las **24 horas** de su creación (`expireAfterSeconds: 86400`).
- **IAJob**: Se auto-elimina a las **24 horas** de su creación (`expireAfterSeconds: 86400`).
- **TrazabilityTemp**: Se auto-elimina a las **48 horas** de su creación (`expireAfterSeconds: 172800`).

5. Prevención de Doble Facturación en Trazabilidad (Diferencia de Índices)

- **TrazabilityHistory (Trazabilidad Definitiva)**: Tiene un **índice compuesto único** sobre `{ lineItem: 1, equipment: 1, serviceDate: 1, requestNo: 1 }`. Esto bloquea a nivel de base de datos cualquier intento de cobrar dos veces un mismo equipo (`equipment`) en la misma fecha (`serviceDate`) bajo la misma partida (`lineItem`) y solicitud presupuestal (`requestNo`).
- **TraceabilityProrratio (Trazabilidad Prorratioada)**: No tiene restricción de unicidad en su índice compuesto (`unique: false`). Esto es por diseño, ya que la trazabilidad prorratioada distribuye intencionalmente el costo de un activo en múltiples celdas o partidas para reflejar el prorratioo contable entre pozos o proyectos.

Detalle Exhaustivo de Directorios de Modelos

1. Actividades de Trabajo (`ActivityModels`)

Representa el desglose cronológico de las tareas realizadas por un equipo o cuadrilla durante su jornada laboral diaria.

- **Archivos:**

- `activity.model.ts` : Esquema de Mongoose y definición de índices.
- `activity.types.ts` : Declaraciones de tipos, interfaces de creación y actualización DTO, y enums de clasificación.

- **Lógica de Negocio y Procesos:**

- **Imputación Horaria:** Cada actividad justifica un bloque de tiempo (definido por `startTime` y `endTime` en formato HH:mm) bajo una tipología de cobro (`activityType` : OP / DISP / MTTO / FS).
- **Asociación de Presupuestos:** Vincula la tarea con claves de contratos: `listTeb` (TEBs oficiales), `listCab` (CABs) o `listCax` (CAXs).
- **Fletes y Kilometraje:** Si la actividad clasifica como `FLETE` (`activityClassification`), se capturan los kilómetros recorridos (`kmQuantity`) y si el viaje fue cargado/plano (`isPlaneLoaded`) o redondo (`isRoundedTrip`).
- **Sobrescritura de Ubicación:** Si la actividad registra un barco (`shipActivie`) o instalación destino específica (`targetInstallationActivie`), estos valores sobrescriben los datos generales del reporte de cabecera al calcular la trazabilidad.

- **Especificación Técnica de Backend:**

- **Estructura del Esquema:**

- `idReport` : `Schema.Types.ObjectId` (ref: "Report" , **requerido**).
- `activityClassification` : `String` (enum: `MOVILIZACION` , `FLETE` , `ARMADO` , "", default: null).
- `activityType` : `String` .
- `kmQuantity` : `Number` .
- `startTime` / `endTime` : `String` ("HH:mm").
- `listCab` / `listCax` / `listTeb` : `[String]` (default: []).
- `isPlaneLoaded` / `isRoundedTrip` / `withCrane` : `Boolean` (default: false).
- `targetInstallationActivie` / `serviceInstallationActivie` / `shipActivie` : `String` (ubicaciones de la actividad).

- `targetInstallationShip: String` (default: `""`).
 - `createAt / updateAt: Date` (default: `Date.now`).
 - **Índices & Reglas:**
 - Índice simple sobre `{ idReport: 1 }` para búsquedas rápidas de las actividades de una bitácora diaria.
 - Timestamps habilitados de Mongoose (`timestamps: true`).
-

2. Banners Informativos (Anuncios)

Permite al administrador enviar notificaciones y alertas de sistema a los paneles de control de los capturistas.

- **Archivos:**
 - `Anuncio.model.ts`: Esquema de Mongoose de la colección `Anuncio`.
 - `types.ts`: Interfaz TypeScript `IAuncio`.
- **Lógica de Negocio:**
 - **Alertas Administrativas:** Difunde mensajes contables masivos (cierres de mes, fechas límites de conciliación de TEBs, mantenimientos del sistema) con prioridades visuales (badge variant).
- **Especificación Técnica:**
 - **Estructura:**
 - `title / description: String` (**requeridos**, auto-trim).
 - `badge: String` (etiqueta superior corta).
 - `badgeVariant: String` (enum: `["default", "success", "warning", "info"]`, default: `"default"`).
 - `ctaText / ctaLink: String` (Call-To-Action para redirigir a manuales o módulos).
 - `variant: String` (enum: `["default", "primary", "accent"]`, default: `"default"`).
 - `icon: String` (clase de icono de interfaz).
 - `active: Boolean` (default: `true`).
 - `expiresAt: Date` (fecha en que el aviso deja de mostrarse).

- **Configuración:** `timestamps: true, versionKey: false` (apaga el campo `__v` de Mongoose).
-

3. Presupuestos Autorizados (`BaseTebModels`)

Almacena el catálogo de autorizaciones presupuestales y órdenes de servicio formales emitidas por el cliente (TEBs/CABs/CAXs).

- **Archivos:**

- `BaseTeb.model.ts` : Esquema y lógica de persistencia.
- `BaseTebTypes.ts` : Interfaces de datos para importación masiva (`TebRecordOne`), DTOs y enums de estado.

- **Lógica de Negocio:**

- **Vínculo Presupuestal:** Es la referencia financiera oficial. Todo cobro en trazabilidad debe estar respaldado por un código de solicitud de servicio válido (`requestService`).
- **Datos Contables:** Contiene los elementos de imputación: programa presupuestal (`budgetProgram`), PEP (`budgetProgram`), EPEP (`element`), Activo (`asset`), pozo (`pool`), y asignación petrolera (`petroleumAssignment`).
- **Flujo de Aprobación:** Los TEBs pasan por los estados: `PENDIENTE` (pendiente de revisión), `APROBADO` (listo para cobro), `RECHAZADO` , o `PENDIENTE_CARGA_DOC` (requiere adjuntar el PDF firmado por el supervisor).
- **Desglose de Materiales:** Detalla los materiales consumibles solicitados en la orden mediante un arreglo de objetos (`materials`).

- **Especificación Técnica:**

- **Estructura:**

- `requestService: String` (**requerido**, indexado simple). Clave única del TEB.
- `dateMadeRequest / dateExecutionService: Date` (**requeridos**).
- `budgetProgram / element / asset / cab / cax / ship / pool / petroleumAssignment: String`.
- `materials: [{ quantity: Number, unity: String, description: String, size: String }]` (default: []).

- `status: String` (enum: `EstatusTeb: PENDIENTE, APROBADO, RECHAZADO, PENDIENTE_CARGA_DOC`, default: `PENDIENTE`).
 - `scanUrl: String` (enlace al PDF cargado en Cloudinary).
 - `documents: [String]`.
 - `createUserInfor / updateUserInfor`: Objetos de auditoría de usuario (`userInfo`).
 - **Índices:**
 - Índice simple `{ createdAt: 1 }` para ordenamiento de auditorías.
 - Índice simple automático sobre `{ requestService: 1 }` por definición de campo.
-

4. Conceptos Parametrizados (`CatalogConceptModels`)

Define los catálogos estandarizados de conceptos contractuales y tipos de documentos permitidos.

- **Archivos:**
 - `catalogConcept.ts`: Interfaz TypeScript `CatalogConcept`.
 - `catalogConcep.type.ts`: Esquema de Mongoose de `CatalogConcept`.
 - `catalogTypeDocuments.ts`: Esquema y tipo para `CatalogTypeDocument`.
- **Lógica de Negocio:**
 - **Estandarización:** Garantiza que las clasificaciones y descripciones de los documentos y actividades se apeguen estrictamente a la lista autorizada en los catálogos del contrato.
- **Especificación Técnica:**
 - `CatalogConceptSchema`:
 - `label: String` (**requerido**). Clave descriptiva del concepto.
 - `createUserInfor / updateUserInfor`: Auditoría.
 - `CatalogTypeDocument`:
 - `typeDocument: String` (**requerido**). Nombre del documento (ej. "Bitácora", "Orden de Flete").
 - `description: String`.

5. Catálogo de Activos y Cuadrillas (CatalogEquipmentsModels)

Contiene la lista oficial de recursos físicos (maquinaria) e integrantes del personal (cuadrillas) asignados al contrato de Nicolás.

- **Archivos:**

- `CatalogEquipment.model.ts` : Esquema del inventario de recursos.
- `CaboManeuverer.model.ts` : Tabla relacional para estructuración de cuadrillas de maniobra.
- `types.ts` : Interfaces y enums de tipos de equipos.

- **Lógica de Negocio:**

- **Inventario del Contrato:** Cada máquina o persona tiene un código único (`code` , ej. `GR-50T-01` para grúas o ficha para personal). Se clasifica por `tipo` (`grua` , `tractocamion` , `cabo` , `maniobrista` , etc.).
- **Jerarquía de Cuadrillas:** Los Cabos actúan como líderes y tienen asignados a un grupo de Maniobristas. Esta asignación es rastreada por `CatalogCaboManeuvererModel` . Al capturar el reporte de un Cabo, el sistema carga automáticamente sus maniobristas asociados.

- **Especificación Técnica:**

- `CatalogEquipment` :
 - `code` : `String` (**requerido**, único).
 - `tipo` : `String` (**requerido**, tipo de recurso).
 - `shortCode` / `description` : `String` .
- `CatalogCaboManeuverer` (Relación):
 - `caboId` : `Schema.Types.ObjectId` (ref: "`CatalogEquipment`", **requerido**).
 - `maneuvererId` : `Schema.Types.ObjectId` (ref: "`CatalogEquipment`", **requerido**).
- **Índices:**
 - **Índice Compuesto Único** en `CatalogCaboManeuverer` : `{ caboId: 1, maneuvererId: 1 }` . Previene la asignación duplicada de un mismo maniobrista al mismo cabo.

6. Catálogos Dinámicos y Logs (`ConfigModels`)

Colección de parámetros del sistema y bitácora de auditoría global del servidor.

- **Archivos:**
 - `CatalogList.model.ts` : Catálogo dinámico llave-valor.
 - `Configurations.model.ts` : Configuraciones del sistema.
 - `Logs.model.ts` : Logs de auditoría de acciones del usuario.
 - `type.ts` : Interfaces de tipado TypeScript de la configuración.
- **Lógica de Negocio:**
 - **Parametrización Dinámica:** Mantiene listas editables de barcos (`ship`), instalaciones destino (`installationDestination`) e instalaciones de servicio.
 - **Seguridad y Auditoría:** El modelo `Logs` graba cada acción sensitiva realizada por los usuarios (inserciones masivas, eliminación de reportes, edición de catálogos).
- **Especificación Técnica:**
 - `CatalogList` :
 - `key` : `String` (**requerido**). Categoría del catálogo.
 - `value` : `String` (**requerido**). Valor almacenado.
 - **Índice Compuesto Único:** `{ key: 1, value: 1 }` para evitar duplicados en un mismo catálogo.
 - `Logs` :
 - `userId` / `dateDay` / `action` / `description` / `module` : **Requeridos**.

7. Perfiles de Base de Datos y Sockets (`ConnectionModels`)

Controla la arquitectura multi-sucursal y los sockets activos para la comunicación en tiempo real.

- **Archivos:**
 - `connection.model.ts` : Conexiones activas de usuarios y sockets.
 - `connection.types.ts` : Tipos y enums del socket.
 - `Profiles.model.ts` : Esquema que asocia oficinas con perfiles de bases de datos.

- **Lógica de Negocio:**
 - **Multi-sucursal:** La base de datos central lee de `Profiles` el URI de MongoDB y credenciales de Cloudinary correspondientes a la oficina (`idOffice`) del capturista para enlazar su sesión con su base de datos aislada.
 - **Sockets en Tiempo Real:** Rastrea la conexión socket (`idSocket`) de cada analista para notificarle vía web sockets los avances en el procesamiento de OCR de bitácoras físicas o la exportación de reportes financieros.
 - **Especificación Técnica:**
 - `Connection` :
 - `profileSelection` : `String` (**requerido**). Perfil seleccionado.
 - `StatusSocket` : `String` (enum `StatusSocketEnum` , **requerido**).
 - `idOffice` : `String` (**requerido**).
 - `IdUser` / `idSocket` : `String` (opcionales).
-

8. Trabajos de IA y Métricas (`DashboardModels`)

Define los umbrales de alarmas presupuestales, las configuraciones de los modelos de lenguaje (Gemini / Claude) y el control de trabajos en segundo plano.

- **Archivos:**
 - `dashboard.model.ts` : Preferencias visuales del capturista.
 - `dashboardMetric.model.ts` : Configuración de alertas de costos y prompts de IA.
 - `dashboardMetric.type.ts` : Enums y tipos de métricas.
 - `iaJob.model.ts` : Trabajos en segundo plano (IA).
 - `type.ts` : Interfaces del dashboard.
- **Lógica de Negocio:**
 - **Control de Trabajos de IA:** Al procesar auditorías masivas de costos, la IA (Gemini) ejecuta análisis que pueden tomar tiempo. El modelo `IAJob` rastrea el estado del trabajo (`pending` , `processing` , `completed` , `failed`), el progreso (0-100%) y almacena el resultado JSON estructurado final.
 - **Alertas y Configuración de Prompts:** `DashboardMetric` centraliza los prompts del sistema (Micro, Macro y Validator) que se enviarán a Gemini, además de definir qué presupuesto máximo dispara notificaciones automáticas por correo electrónico.

- **Estado de la UI:** `DashboardDate` persiste las preferencias del analista en el navegador, guardando qué acordeones de información tiene colapsados o el nivel de scroll del panel.
 - **Especificación Técnica:**
 - `IAJob` :
 - `jobId: String` (**requerido**, único, indexado).
 - `type: String` (enum `['micro', 'macro']`, **requerido**).
 - `status: String` (default `pending`).
 - `requestData / result: Schema.Types.Mixed` (datos flexibles de entrada y salida).
 - **Índice TTL:** `{ createdAt: 1 }` con `expireAfterSeconds: 86400` (24 horas). Limpia de forma autónoma el historial de trabajos para ahorrar almacenamiento.
 - `DashboardMetric` :
 - `title: String` (**requerido**).
 - `aiConfig` : Objeto con configuraciones específicas de modelos y prompts para: `micro`, `macro` y `validator`.
-

9. Partidas Contractuales e Historial Operativo (`DepartureModels`)

Este es el núcleo de cálculo del sistema. Vincula las partidas del contrato con los activos del catálogo, registrando el conteo de días trabajados y mantenimientos.

- **Archivos:**
 - `departure.model.ts` : Esquema de la Partida contractual.
 - `subitems.model.ts` : Relación entre Partida y Equipo.
 - `subItemLaborDay.model.ts` : Registro diario de la jornada de un equipo.
 - `relationDepartureEquipmentType.model.ts` : Tabla pivote de asignación de equipos y reportes.
 - `reportDailySubitems.model.ts` : Relación directa entre Bitácoras y Sub-items.
 - `departure.types.ts` : Interfaces de cálculo, tipos de cobro y enums.
- **Lógica de Negocio y Procesos de Cálculo:**

- **Partida Contractual (`Departure`)**: Modela una línea del contrato (ej. "Partida 2.1 - Grúa de 50T"). Define la tarifa (`unitCost`), tipo de moneda (`currency`), y banderas de modalidad de cobro: flete (`isShipping`), cobro por día (`isWorkDayInOperation`), armado (`isAssembly`), cobro por hora (`isHourInOperation`), disponibilidad (`isAvailable`), o cobro por evento (`isByService`).
- **Sub-item (`SubItems`)**: Representa la asignación física de una máquina específica (`CatalogEquipment`) a una Partida. Soporta el flag `isOutside: true` (si el activo trabaja fuera del muelle principal, lo cual aplica un descuento reglamentario del 75% sobre la tarifa diaria).
- **Jornadas de Trabajo (`SubItemLaborDay`)**: Almacena el valor acumulado diario de cobro.
 - `value` : Fracción decimal diaria del activo (0 a 1.0).
 - `valueRounded` : Decimal ajustado después del redondeo del turno.
 - `amountEstimated` / `amountMaintenance` : Importe monetario computado.
 - `activitiesBreakdown` : Desglose proporcional que detalla qué actividades del día consumieron qué fracción del costo, vinculándolas a sus respectivos códigos TEB, CAB y CAX.
- **Especificación Técnica:**
 - `Departure` :
 - `code` : `String` (**requerido**, único).
 - `typeDeparture` : `String` (tipo de activo: `grua` , `maniobrista` , etc., **requerido**).
 - `unitCost` : `Number` (default 0).
 - `allowedEquipments` : `[Schema.Types.ObjectId]` (ref: `"CatalogEquipment"`).
 - `subItems` : `[Schema.Types.ObjectId]` (ref: `"SubItems"`).
 - `SubItems` & `Virtuals`:
 - `idDeparture` / `idCatalogEquipment` : `ObjectIds` (**requeridos**).
 - **Virtual `laborDays`** : Enlaza dinámicamente con `SubItemLaborDay` filtrando los días operativos (`isMaintenance: false`).
 - **Virtual `laborDaysMaintenance`** : Enlaza con `SubItemLaborDay` para días de mantenimiento (`isMaintenance: true`).

- **Pre-find Hook:** Ejecuta de manera nativa un middleware de Mongoose que autodeclara `.populate('laborDays')` y `.populate('laborDaysMaintenance')` en cada consulta `find / findOne` para simplificar el código del servicio.
 - `SubItemLaborDay`:
 - `subItemId: Schema.Types.ObjectId (ref: "SubItems", requerido, indexado)`.
 - `day: Date (requerido)`.
 - `isMaintenance: Boolean (requerido, indexado)`.
 - **Índice Compuesto:** `{ subItemId: 1, isMaintenance: 1, day: 1 }` para agilizar agregaciones de costos por periodos de tiempo.
-

10. Bitácora de Edición Manual (`EditModeModels`)

Registra las acciones de edición manual directa ejecutadas por usuarios con privilegios de administrador en datos históricos de trazabilidad.

- **Archivos:**
 - `editMode.model.ts`: Esquema de Mongoose.
 - `editMode.type.ts`: Tipo de TypeScript de los registros de edición.
 - **Lógica de Negocio:**
 - **Auditoría de Modificaciones:** Cuando un administrador fuerza el cambio de una celda contable definitiva de trazabilidad, el sistema le obliga a registrar la justificación y almacena el valor anterior, el nuevo valor, el usuario y la fecha del cambio.
 - **Especificación Técnica:**
 - `EditMode`:
 - `typeModification: String (requerido)`. Categoría de edición.
 - `actualDate: Date (requerido)`.
 - `descriptionModification / nameProperty / descriptionGeneral: String (requeridos)`.
-

11. Costos Estimados Cache (`EstimatedCostModels`)

Tabla de optimización contable que almacena los costos computados de manera estática.

- **Archivos:**
 - `estimatedCost.model.ts` : Esquema de persistencia.
 - `type.ts` : Tipo `EstimatedCost`.
 - **Lógica de Negocio:**
 - **Desempeño:** Evita que las pantallas del Dashboard y reportes mensuales calculen en tiempo real fórmulas de costo complejas (que involucran cruces de Sub-items, descuentos por factor externo del 75%, y proporciones horarias). Los resultados se escriben en esta cache para consultas inmediatas.
 - **Especificación Técnica:**
 - `EstimatedCost` :
 - `subItemId / laborDaysId : String` (**requeridos**). Identificadores de cruce.
 - `isMaintenance : Boolean`.
 - `estimatedCost : Number` (costo monetario almacenado).
-

12. Tareas de Exportación Asíncrona (`ExportTaskModel`)

Rastrea el progreso de la generación masiva de archivos de Excel para evitar el bloqueo del event loop en el backend.

- **Archivos:**
 - `ExportTask.model.ts` : Esquema de Mongoose.
 - `types.ts` : Interfaz `IExportTask`.
- **Lógica de Negocio:**
 - **Procesamiento Asíncrono:** La generación de Excel de trazabilidad con miles de filas es delegada a un subprocesso. Esta colección expone al frontend el estado actual de la tarea (`status : PENDING, PROCESSING, COMPLETED, FAILED`) y el link final de descarga en Cloudinary (`fileUrl`).
- **Especificación Técnica:**
 - `ExportTask` :
 - `type : String` (ej. `"traceability"` , **requerido**).
 - `status : String` (enum, default `"PENDING"`).
 - `progress : Number` (0 a 100).

- `userId: String` (**requerido**).
 - `filters: Schema.Types.Mixed` (filtros de fecha y sucursales usados).
 - `fileUrl / error: String`.
-

13. TEBs Provisionales y Pivote (`FictitiousTeb`)

Permite capturar reportes en campo bajo claves presupuestales ficticias cuando el cliente tiene demoras en emitir el TEB oficial.

- **Archivos:**
 - `fictitiousTeb.model.ts` : Contiene dos esquemas: `FictitiousTeb` y el pivote relacional `FictitiousTebPivot`.
 - `type.ts` : Enums y tipos de TEBs ficticios.
- **Lógica de Negocio y Procesos:**
 - **Contingencia Operativa:** Evita detener el registro de actividades en campo. El capturista crea un "TEB Ficticio" que el sistema registra con un folio autonumérico incremental con prefijo `TEBF`.
 - **Cierre y Conciliación (Pivote):** Cuando el cliente entrega el documento definitivo con la clave real, el administrador asocia el TEB Ficticio con el TEB Real (`BaseTeb`). `FictitiousTebPivot` mapea este enlace, permitiendo actualizar en cascada todas las filas de trazabilidad que estaban apuntando al folio provisional.
- **Especificación Técnica:**
 - `FictitiousTeb` :
 - `folio: String` (**único**, generado automáticamente).
 - `dateMadeRequest / dateExecutionService: Date` (**requeridos**).
 - `status: String` (enum `FictitiousTebStatus`: `CREATED`, `LINKED`, `CLOSED`, default: `CREATED`).
 - `labels: [Map]` (metadatos asociados).
 - **Pre-save Hook:** Mongoose autoincrementa el consecutivo en `SequenceFolioModel` bajo la clave `{ table: "FictitiousTeb" }` y genera un folio con formato `TEBF0000000001` (rellenado con ceros a la izquierda hasta 10 posiciones).
 - `FictitiousTebPivot` :

- `idBaseTeb : ref "BaseTeb" .`
 - `idFictitiousTeb : ref "FictitiousTeb" .`
 - `idReport : ref "Report" .`
 - `idActivityReport : ref "ActivityReport" .`
-

14. Catálogo de Turnos Contractuales (`JornDayModels`)

Define los turnos oficiales autorizados en el contrato y las ventanas horarias de validez.

- **Archivos:**
 - `jornDay.model.ts` : Esquema de Mongoose.
 - `jornDay.types.ts` : Interfaces de turnos y DTOs de creación.
 - **Lógica de Negocio:**
 - **Redondeo Contractual:** Modela turnos como "Matutino", "Diurno", "Nocturno", etc. En el arreglo `dividedHrsByDay.listHrsDay` , se definen los sub-intervalos y se especifica cuál de ellos es el de ajuste (`isAdjustment : true`). Este slot asimilará las imprecisiones de los decimales de las fracciones de día calculadas para evitar que la suma final del día difiera de `1.00` .
 - **Especificación Técnica:**
 - `JornDay` :
 - `description : String` (**requerido**, único).
 - `duration : Number` (**requerido**).
 - `dayEsp : String` (**requerido**).
 - `startTime / endTime : String` ("HH:mm", **requeridos**).
 - `dividedHrsByDay` : Objeto con:
 - `isDivided : Boolean` .
 - `hrsCountDay : Number` .
 - `listHrsDay` : Arreglo de sub-turnos { `startTime : String`, `endTime : String`, `isAdjustment : Boolean` }.
-

15. Estructura de Auditoría de Usuario (`RegisterInfo`)

- **Archivos:**

- `saveInfo.ts` : Interfaz común `userInfo` (`userId` , `userName` , `emailUser`). Usado para el tipado estricto del autor y editor en todos los esquemas.
-

16. Cabecera del Reporte Diario (`ReportModels`)

Representa el documento maestro físico del reporte diario capturado en campo para un activo o cuadrilla de trabajo.

- **Archivos:**

- `report.model.ts` : Esquema Mongoose con la lógica de folios y guardado.
- `report.types.ts` : Estructuras de datos, interfaces de validación de volumen y DTOs.

- **Lógica de Negocio y Reglas Críticas:**

- **Bitácora Única:** Un reporte documenta la jornada de un recurso (`assetNumber`) en una fecha específica (`dailyReportDate`) y turno (`shiftType`).
- **Validación de Jornada Máxima:** La suma de las variables del día (`valueOperation` + `valueAvailable` + `valueMaintenance` + `valueOutOfService`) representa la fracción de cobro diaria acumulada por el equipo, la cual **nunca** debe superar `1.00` .
- **Regla de Cabos:** Si el reporte es de tipo Cabos (`caboServiceDay` : `true`), el arreglo `caboIds` debe registrar un máximo de 1 Cabo.

- **Especificación Técnica:**

- `Report` :
 - `folio` : `Number` (**único**, consecutivo automático).
 - `caboServiceDay` / `reportFleet` / `maniobristaServiceDay` : `Boolean` (default `false`).
 - `assetNumber` : `String` (identificador económico del activo).
 - `dailyReportDate` : `Date` (**requerido**).
 - `shiftType` : `String` .
 - `caboIds` / `maniobristaIds` : `[String]` .
 - `targetInstallation` / `serviceInstallation` / `ship` : `String` .
 - `valueOperation` / `valueAvailable` / `valueMaintenance` / `valueOutOfService` : `Number` .

- **Índices & Reglas:**
 - Índice simple sobre `{ dailyReportDate: 1 }` para agrupar reportes por día.
 - **Pre-save Hook:** Al insertar un nuevo reporte, Mongoose consulta atómicamente a `SequenceFolioModel` bajo la clave `{ table: 'Report' }` para incrementar en 1 el contador del folio y guardarlo en el documento.
-

17. Atributos de Capacidad (`ResourceModels`)

Define las limitantes contractuales de capacidad de los activos pesados en el contrato.

- **Archivos:**
 - `resource.model.ts` : Esquema de Mongoose.
 - `resource.types.ts` : Tipos del recurso.
 - **Lógica de Negocio:**
 - **Validación de Fletes y Grúas:** Cataloga la capacidad de carga (ej. toneladas de grúa, capacidad de arrastre) para validar si el equipo es apto para ejecutar fletes específicos.
 - **Especificación Técnica:**
 - `Resource` :
 - `catalog` : Objeto embebido con `{ code: String (único), tipo: String, description: String }`.
 - `capacity` : `String` (**requerido**).
 - `unit / specification` : `String` (**requeridos**).
 - `departures` : `[{ type: Schema.Types.ObjectId, ref: "Departure" }]`.
-

18. Sesiones de Escaneo OCR (`ScannerModels`)

Controla la sesión interactiva del analista al procesar lotes masivos de PDFs para lectura con inteligencia artificial.

- **Archivos:**
 - `daily-report-session.model.ts` : Esquema con la lógica TTL para sesiones.
- **Lógica de Negocio:**

- **OCR Asíncrono:** Controla el porcentaje de progreso (`processedFiles` / `totalFiles`) de digitalización del lote de imágenes analizadas por Gemini, guardando temporalmente los errores y aciertos de interpretación.
 - **Especificación Técnica:**
 - `DailyReportScannerSession` :
 - `sessionId` / `userId`: String (**requeridos**, indexados).
 - `status`: String (enum: ['processing', 'completed', 'failed', 'partially_completed'], default processing).
 - `totalFiles` / `processedFiles`: Number (default 0).
 - `results`: [Schema.Types.Mixed] .
 - `processingErrors`: [{ `fileName`: String, `error`: String }].
 - **Índice TTL de Limpieza:**
`DailyReportScannerSessionSchema.index({ createdAt: 1 }, { expireAfterSeconds: 86400 });` (auto-destrucción en **24 horas**).
-

19. Secuencias Atómicas de Consecutivos (`SequenceFolioModels`)

- **Archivos:**
 - `SequenceFolioModels.model.ts` : Esquema simple de contador (`folio`: Number , `table`: String). Utilizado por los middlewares pre-save de las colecciones que requieren folios enteros ordenados correlativamente.
-

20. Capturas Temporales del OCR (`TempModels`)

Almacena de forma provisional la información interpretada por los modelos de visión artificial antes de ser autorizados.

- **Archivos:**
 - `temporaryDailyReports.model.ts` : Esquema de Mongoose.
 - `temporary.type.ts` : Interfaces de tipado del OCR.
- **Lógica de Negocio y Procesos:**

- **Previsualización:** Al subir las fotos de las bitácoras físicas, Gemini extrae las actividades y detecta el número económico del activo. Debido a que las bitácoras físicas suelen tener tachaduras o mala caligrafía, el sistema realiza búsquedas difusas (fuzzy search) y guarda en `suggestedEquipment` una lista de activos candidatos ordenados por similitud (`similarity`). El analista debe confirmar en pantalla dividida qué activo es el real antes de guardarlo en la base definitiva.
 - **Especificación Técnica:**
 - `TemporaryDailyReport` :
 - `dailyReportDate` / `assetNumber` / `assetNumberOriginal` / `shiftType` : `String` .
 - `suggestedEquipment` : Arreglo de objetos { `id`, `code`, `description`, `similarity`: `Number` } .
 - `caboServiceDay` : `Boolean` .
 - `caboIds` / `caboNames` : `[String]` .
 - `activities` : Sub-esquema con el desglose leído de actividades (`startTime`, `endTime`, `listTeb`, `listCab`, etc.).
 - `documentUrl` : `String` de la imagen de la página.
 - `warnings` : `[String]` (alertas de inconsistencias en la lectura).
 - `confidence` : `Number` (nivel de certeza del OCR).
 - `officeId` / `socketId` : **Requeridos**. Usados para enviar las correcciones del OCR al socket del navegador que subió el lote.
-

21. Historial de Trazabilidad Real y Prorrateada (`TrazabilityHistoryModel`)

Tabla final consolidada que representa la exportación financiera definitiva y de auditoría contable.

- **Archivos:**
 - `TrazabilityHistoryModel.model.ts` : Esquema de trazabilidad cerrada.
 - `TraceabilityProrratio.model.ts` : Esquema de distribución prorratio.
 - `types.ts` : Declaraciones de tipos TypeScript (`ITrazabilityType` , `ITraceabilityProrratioType`).

- **Lógica de Negocio:**

- **Registro Financiero Final:** Es el destino de todos los cálculos mensuales. Cada fila representa un importe monetario asignado a una fecha, activo, partida e imputación contable detallada (PEP, EPEP, CeGe).
- **Auditoría de Modificaciones:** El campo `modifiedBy` almacena el histórico de quién, cuándo y qué columnas editó manualmente sobre esta fila una vez que fue consolidada.

- **Especificación Técnica:**

- `TrazabilityHistory` (Trazabilidad Definitiva):
 - `workType / month / contract / lineItem / equipment / serviceType / serviceDescription / requestNo / installation / pep / ep / asset / raf / serviceOrigin / serviceDestination: String (requeridos).`
 - `estimationNo / unitPrice / amount / volume: Number (requeridos).`
 - `modifiedBy: Arreglo de cambios { name: String, fields: Array, modifiedAt: Date }.`
 - **Índice Compuesto Único:**

```
TrazabilityHistorySchema.index(  
  { lineItem: 1, equipment: 1, serviceDate: 1, requestNo: 1 },  
  { unique: true }  
);
```

Garantiza que no exista doble cobro de un mismo equipo en el mismo día, bajo la misma partida y el mismo TEB.

- `TraceabilityProrratio` (Trazabilidad Prorratio):
 - Misma estructura de campos que la trazabilidad definitiva.
 - **Índice Compuesto No Único:**

```
TraceabilityProrratioSchema.index(  
  { lineItem: 1, equipment: 1, serviceDate: 1, requestNo: 1 },  
  { unique: false }  
);
```

Permite la coexistencia de múltiples registros para el mismo activo/fecha, ya que representa la fragmentación prorrateada del costo diario entre distintos pozos o instalaciones del cliente.

22. Sesión Temporal de Trazabilidad (`TrazabilityTempModel`)

Colección de almacenamiento temporal que actúa como borrador al realizar simulaciones de estimaciones de costo.

- **Archivos:**
 - `TrazabilityTempModel.model.ts`: Esquema de Mongoose.
- **Lógica de Negocio:**
 - **Sandbox de Simulación:** Antes de consolidar la trazabilidad final del mes, el analista puede correr simulaciones. El sistema escribe las filas preliminares en esta colección bajo un `sessionId`. Si la simulación es correcta, se confirma (`sessionStatus`: `'SAVED'`) y se vuelcan los datos a `TrazabilityHistory`. Si se descarta, se borran.
- **Especificación Técnica:**
 - `TrazabilityTemp`:
 - `sessionId / userId`: String (**requeridos**, indexados).
 - `sessionStatus`: String (enum: `['PENDING', 'SAVED', 'DISCARDED']`, default `'PENDING'`).
 - `dateFrom / dateTo`: String (**requeridos**).
 - `isSimulated`: Boolean (default `false`).
 - `record`: `Schema.Types.Mixed` (**requerido**). Estructura idéntica al registro de trazabilidad.
 - **Índice TTL de Autolimpieza:**
`TrazabilityTempSchema.index({ createdAt: 1 }, { expireAfterSeconds: 172800 } )`; (auto-eliminación en **48 horas**).